

Stochastic Anomaly Simulator

Project Documentation

Pole Anomaly Project

May 22, 2026

Contents

1	Introduction	2
2	Quick Setup Guide	3
2.1	Environment Setup	3
2.2	Dependencies	3
3	Synthetic Data Generation Pipeline (data-gen)	4
3.1	File Structure Review	4
3.2	Explanation of the Engine and Scripts	4
3.2.1	data-gen/anomaly_generator.py	4
3.2.2	data-gen/evaluate_real_data.py	4
3.2.3	data-gen/interactive_labeling_helper.py	4
3.2.4	data-gen/generate_custom_dataset.py	5
3.2.5	data-gen/test_generator_deformation.py	5
3.2.6	data-gen/visualize_anomalies_templates.py	6
3.3	Anomaly Template Creation	6
3.3.1	Configuration Blueprint (CSV Logic)	6
4	Inference and Evaluation Methodology	8
4.1	Window-Level Evaluation (Training Script)	8
4.2	Event-Level Evaluation (Inference Script)	8

1 Introduction

This document provides a comprehensive overview of the **Pole Anomaly Project**, an anomaly detection and classification system for nuclear gamma dosimetry signals. The overall objective is to automatically identify and classify transient anomalous events—such as radiation spikes with distinct geometric signatures—embedded within continuous, noisy gamma dose rate measurements.

The project follows a two-phase approach. First, a **synthetic data generation pipeline** produces large-scale training datasets by injecting user-defined anomaly templates (Bell curves, Squares, M-shapes, Exponential ramps, etc.) into realistic simulated radiation backgrounds derived from real-world sensor baselines. Second, multiple **1D Deep Learning architectures (1D-CNN, ResNet, CNN-Attention)** are trained on these synthetic datasets to perform multi-class anomaly classification in a streaming inference setting.

The repository is organized into three main components:

- **data-gen/**: The synthetic data generation pipeline, including anomaly templates, the stochastic generator engine, and visualization scripts.
- **architectures/**: The neural network implementations, including the model architectures (1D-CNN, ResNet, CNN-Attention), training loop, inference engine, and robustness testing scripts.
- **documentation/**: This document and associated media.

2 Quick Setup Guide

This project isolates dependencies using a Python virtual environment to ensure absolute reproducibility.

2.1 Environment Setup

To initialize the project on a new machine, execute the following commands in the root of the project:

```
# 1. Create a Python Virtual Environment
py -3.14 -m venv .venv

# 2. Activate the Environment (Windows PowerShell)
.\.venv\Scripts\Activate.ps1

# 3. Install all dependencies
pip install -r requirements.txt
```

2.2 Dependencies

The `requirements.txt` installs the following libraries:

- `numpy` & `pandas`: Fast numerical array manipulation and CSV template handling.
- `torch` (PyTorch): Deep learning framework for the 1D-CNN. Installed from the PyTorch Nightly channel with CUDA 12.8 support via `--extra-index-url`.
- `plotly` & `kaleido`: Interactive HTML visualizations and static PNG export.
- `scikit-learn`: Utility functions for metrics and data splitting.
- `matplotlib`: Supplementary plotting library.
- `scipy`, `statsmodels`, `tslearn`: Statistical interpolation and legacy dataset compatibility.

Note: The `--pre` flag in `requirements.txt` enables installation of pre-release PyTorch builds, which is required for CUDA 12.8 support on newer GPUs (e.g., NVIDIA RTX 5060).

3 Synthetic Data Generation Pipeline (data-gen)

As the repository expands to include other projects (such as neural networks for anomaly detection), this section specifically details the anomaly generation pipeline.

3.1 File Structure Review

The repository enforces a strict separation of concerns, divided into modular directories:

- `data-gen/data/`: Contains raw background sensor readings (e.g., `2015_months_DebitDoseA.txt`) used as a real-world baseline, along with generated output datasets and the class mapping file (`class_mapping.txt`).
- `data-gen/anomalies/`: Contains the normalized `.csv` blueprints (templates) for the 6 anomaly shapes: `bell`, `bell_sq`, `fast_ascend`, `fast_descend`, `M`, and `square`.
- `data-gen/logs/`: The destination for Plotly’s interactive HTML visualizations and PNG exports.
- `data-gen/`: Stores the pure mathematical generator library (`anomaly_generator.py`) alongside execution pipelines, real-data evaluators, and visualization launchers.

3.2 Explanation of the Engine and Scripts

3.2.1 `data-gen/anomaly_generator.py`

This is the core computational engine. It provides the `generate_anomaly()` function which transforms a normalized CSV blueprint into a physical discrete array. The engine mathematically scales the template temporally (period span) and vertically (amplitude intensity) while embedding Gaussian deformation using the limits declared in the CSV file.

3.2.2 `data-gen/evaluate_real_data.py`

This script analyzes the original physical gamma sensor data to extract baseline drift and noise characteristics. It calculates the moving average baseline to isolate high-frequency noise from the slow, low-frequency sensor drift. Crucially, it calculates the volatility σ of the Ornstein-Uhlenbeck (OU) process used for synthetic generation. Because a moving average heavily dampens step-by-step differences, the script applies a calibrated mathematical correction to σ (e.g., 0.037) so that the synthetic baseline explores the same dynamic range (15-27 units over 40,000 points) as the real 2015 sensors. The results are saved to `real_data_params.json`.

3.2.3 `data-gen/interactive_labeling_helper.py`

An interactive visualization tool that plots the real, unlabelled sensor data alongside its calculated baseline and a 3σ confidence threshold. It allows the user to hover over visual anomalies in the real data to measure their exact amplitude and period, which directly informs the configuration parameters for synthetic generation.

3.2.4 data-gen/generate_custom_dataset.py

The primary execution pipeline for training data. It utilizes a centralized `CONFIG` dictionary explicitly defining generation constraints (such as anomaly amplitude, period, and frequency) and seamlessly imports the real-world statistical parameters generated by `evaluate_real_data.py`.

Spanning datasets of up to 10,000,000 steps, it generates a highly realistic **wandering baseline** using an Ornstein-Uhlenbeck (OU) process.

The Wandering Baseline Logic (OU Process): Real-world sensors (like Gamma dose monitors) rarely maintain a perfectly flat baseline; they experience slow, low-frequency drifts over time due to environmental factors like temperature changes or sensor calibration drift. To mathematically simulate this, the generator employs the Ornstein-Uhlenbeck process, a mean-reverting random walk described by the discrete stochastic differential equation:

$$y_t = y_{t-1} + \theta(\mu - y_{t-1}) + \sigma\mathcal{N}(0, 1) \quad (1)$$

Where:

- μ (Global Mean): The long-term equilibrium value (e.g., 99.26).
- θ (Mean-Reversion Rate): Controls how strongly the drift is pulled back to the mean (e.g., 0.000004, ensuring very slow, realistic wandering).
- σ (Volatility): The scale of the random drift step.

This wandering baseline is then overlaid with standard high-frequency Gaussian noise. Finally, it randomly samples anomaly templates from the library, calculates randomized injection amplitudes and duration bounds, and injects them into the signal. Each injected shape is mapped to a unique multi-class integer ID, tagging absolute ground-truth labels alongside the continuous signal.

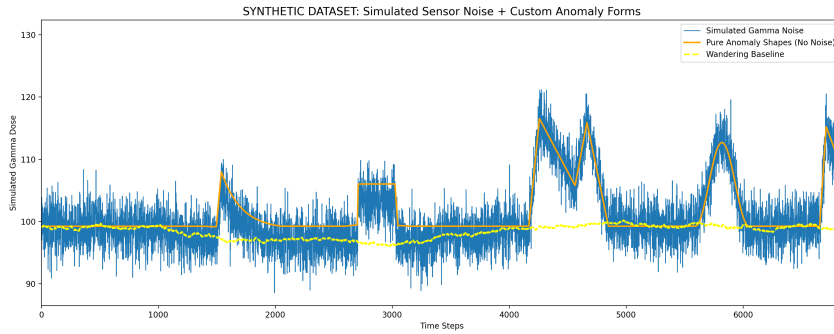


Figure 1: Output of `generate_custom_dataset.py`

3.2.5 data-gen/test_generator_deformation.py

A visual sandbox and boundary debugging tool. Iterating automatically over all discovered templates in the `data-gen/anomalies/` directory, it runs exactly 5 generation iterations of each template to visually verify and validate the variability limits enforced by the template's deformation parameters. Exports directly to HTML and PNG.

3.2.6 data-gen/visualize_anomalies_templates.py

This script visualizes the pure original mathematical blueprints. It renders the shapes and their interpolation properties exactly as described in the blueprint CSV, without injecting randomization or temporal noise.

3.3 Anomaly Template Creation

Instead of relying on rigid pre-recorded dataset distributions, targeted geometric structures are freely defined as CSV blueprints in the `data-gen/anomalies/` folder.

3.3.1 Configuration Blueprint (CSV Logic)

A standard anomaly template requires 7 strict column definitions mapping normalized coordinates and behavioral constraints:

`time, value, t_minus, t_plus, v_minus, v_plus, interp`

- **time & value:** Normalized shape coordinates (scaling from 0.0 to 1.0). They define the key inflection points of the curve.
- **Deformation Multipliers (`t_minus`, `t_plus`, `v_minus`, `v_plus`):** Define spatial jitter limits per control point. A value of 1.0 allows symmetric Gaussian distortion up to 100% of the global variance parameter. A value of 0.0 locks the point rigidly in that direction.
- **interp (Interpolation Mode):** Defines the mathematical connection between consecutive control points:
 - **linear:** Simple linear interpolation between points.
 - **exp_up:** Concave-upward exponential growth (slow start, rapid finish), with curvature controlled by $\alpha = 3.0$.
 - **exp_down:** Concave-downward exponential curve (rapid start, slow finish), the complement of **exp_up**.
 - **bell:** Cosine envelope ($\frac{1}{2} - \frac{1}{2} \cos(\pi x)$), generating a smooth S-curve suitable for Gaussian-like bell shapes.

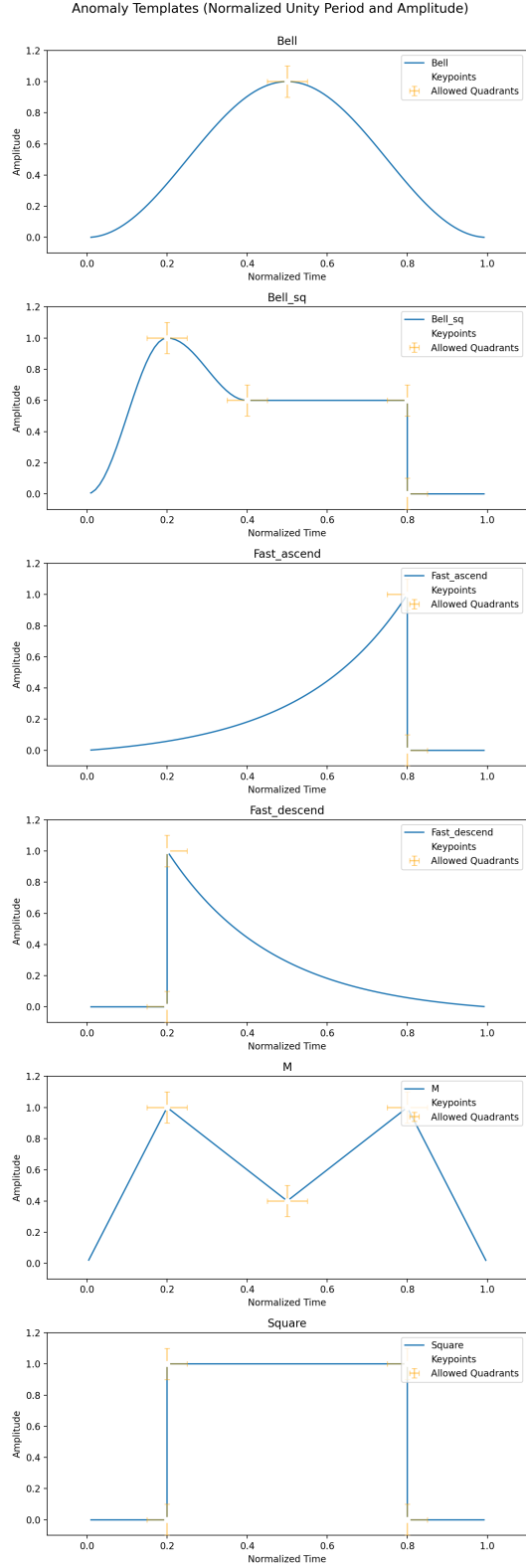


Figure 2: Output of visualize_anomalies_templates.py

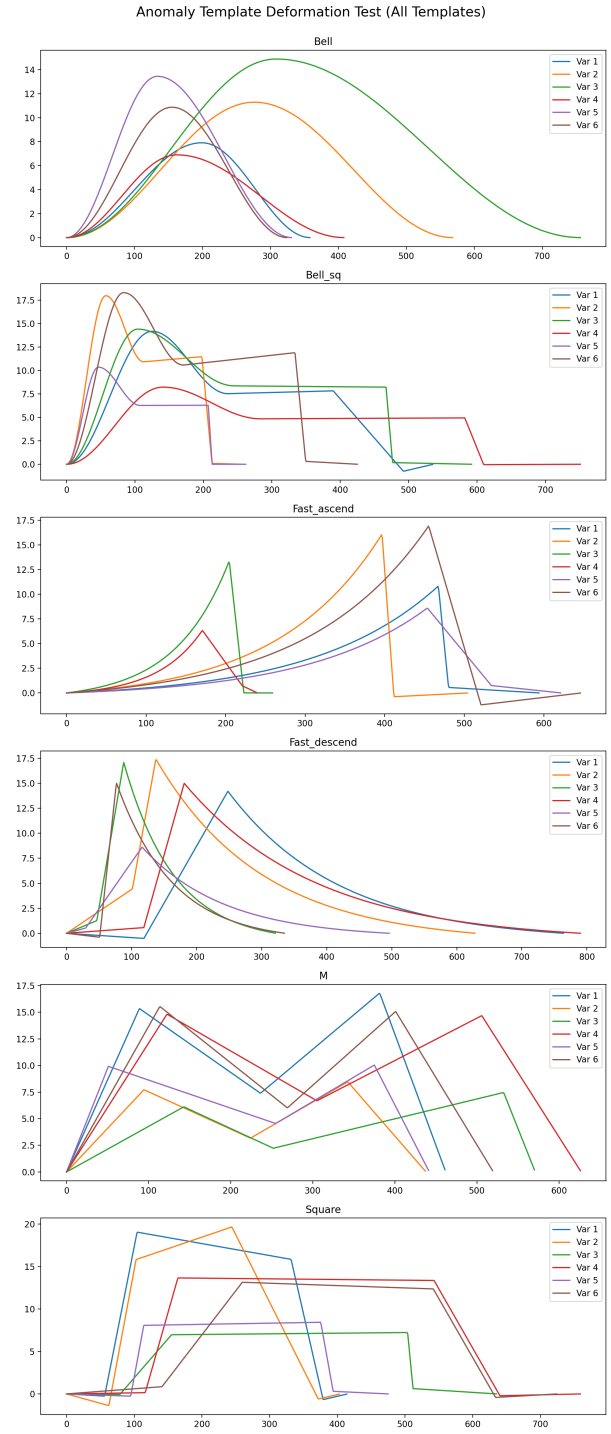


Figure 3: Output of test_generator_deformation.py

4 Inference and Evaluation Methodology

Anomaly detection models on time series data face a fundamental discrepancy between how they are *trained* and how their real-world performance is *evaluated*. This project employs two distinct evaluation paradigms: Window-Level Evaluation (used strictly during training) and Event-Level Evaluation (used during inference and robustness testing).

4.1 Window-Level Evaluation (Training Script)

During training (e.g., in `train.py`), the continuous time series is sliced into thousands of overlapping windows of a fixed size (e.g., 512 points). A window is labeled as an anomaly if anomalous points occupy more than a strict threshold (e.g., 10%) of the window's span.

F1 Score Penalty: Because the dataset relies on sliding windows, many generated windows capture only the extreme edges of an anomaly (e.g., 85% background noise and 15% anomaly curve). These "boundary" windows are highly ambiguous and often misclassified by the network. During training, the Macro F1 score evaluates each window independently. Consequently, the F1 score appears artificially deflated because the network is heavily penalized for hesitating on these blurry edges, especially under severe class imbalance conditions.

4.2 Event-Level Evaluation (Inference Script)

To provide a more practical and realistic assessment of model performance, the inference scripts (such as `visualize_inference.py` and `robustness_test.py`) employ Event-Level Evaluation.

Methodology:

1. **Accumulation:** As the sliding window passes over the continuous data, the model predicts the probabilities for each window. Since the windows overlap (stride of 10), each individual time step receives multiple probability predictions. These predictions are accumulated and averaged, creating a smoothed, continuous probability curve for the entire dataset.
2. **Event Grouping:** Instead of judging rigid windows, the script groups contiguous ground-truth anomaly points into a single discrete "event."
3. **Scoring:** An event is considered successfully detected and correctly classified if the averaged prediction curve peaks at the correct class at any point during the event's duration.

Impact on Metrics: By grouping points into events, the model is no longer penalized for boundary ambiguity. As long as the network's confidence correctly identifies the anomaly at its core/peak, the entire event is counted as a success. This leads to near 100% accuracy in inference, accurately reflecting the model's practical utility despite the deflated window-level F1 scores seen during training.